

AI Security 2.4. AI System Reconnaissance

Task 1 - Introduction

Instead of modelling threats, we go looking for the actual AI systems on a live network. The two skills are complementary but distinct: threat modelling works with assumptions about what exists, while reconnaissance confirms what is actually deployed and exposed. When organisations add AI capabilities, their networks change in ways that most security teams never see. New services appear on unfamiliar ports. APIs respond in formats that standard scanners do not recognise. ML platforms, model registries, and experiment-tracking servers are deployed with default configurations that prioritise ease of use over security.

What is AI Reconnaissance: AI reconnaissance is the practice of identifying these components, understanding what they are, and what they expose. You still use tools like Nmap and curl, but the targets are different. Standard scanning wordlists and service detection miss AI infrastructure entirely because they were not built to find it.

Why This Matters Right Now: In January 2026, security researcher Maor Dayan ran a targeted Shodan scan and found 42,665 exposed AI agent instances([opens in new tab](#)) on the public internet. 93.4% were vulnerable, and many were leaking API keys through unauthenticated connections. Around the same time, GreyNoise captured over 91,000 attack sessions targeting AI deployments in three months. Attackers are already scanning for AI infrastructure at scale. Traditional tools miss it entirely.

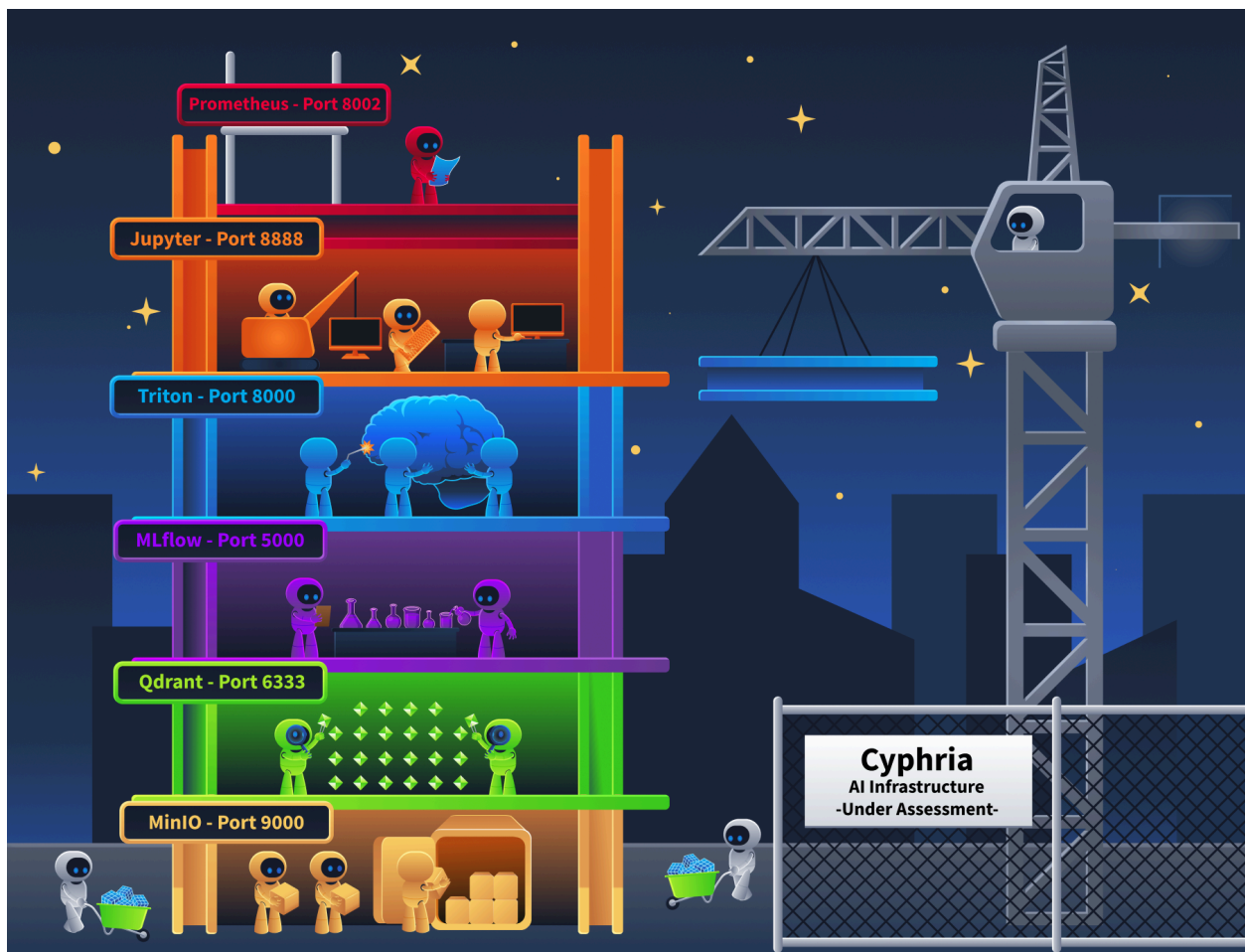
Learning Objectives

This room focuses on AI infrastructure reconnaissance. Other rooms in the module cover architectural design, threat modelling, and vulnerability assessment. Here, we stick to discovery and enumeration. By the end, you will be able to:

- Identify the common components of a production AI/ML system and the ports, protocols, and API paths each one exposes
- Fingerprint AI and ML services by analysing HTTP headers, API response structures, error messages, and endpoint naming conventions
- Enumerate model metadata, experiment histories, and deployment configurations from exposed tracking servers and inference endpoints

- Map how AI components expand the traditional network attack surface, including new data flows, internal APIs, and supply chain dependencies
- Apply a structured AI reconnaissance methodology using existing security tools with AI specific configurations
- Recognise the logging and monitoring signatures that indicate AI focused reconnaissance activity against your infrastructure

Task 2 - The AI Infrastructure Stack



A production AI system is not a single server. It is a collection of specialised services that handle different parts of the machine learning lifecycle. Here is what you will encounter during reconnaissance.

Model Serving Endpoints: These are the frameworks that load trained models into memory and expose prediction APIs. They are the front door of any AI deployment.

- NVIDIA Triton Inference Server is one of the most common. It exposes HTTP on port 8000, gRPC on port 8001, and Prometheus metrics on port 8002. Three ports for one service.
- TensorFlow Serving uses gRPC on 8500 and HTTP on 8501.
- TorchServe exposes inference on 8080, a management API on 8081, and metrics on 8082.
- For LLM specific deployments, Ollama runs on port 11434, and vLLM typically serves on port 8000. Both expose OpenAI-compatible API formats.

Why do many of these expose both HTTP and gRPC at the same time? gRPC is faster for streaming large tensor payloads between internal services. HTTP/REST is easier for client applications to consume. So they run both.

Orchestration and Experiment Tracking: These platforms manage the entire ML lifecycle, from experiment design through to model deployment. They are the **highest-value targets** during reconnaissance because they contain everything.

- MLflow Tracking Server runs on port 5000.
- It records every experiment, every hyperparameter configuration, every training metric, and every model artifact the organisation produces.
- If you find an open MLflow instance, you have found their complete ML research history.
- Kubeflow runs on Kubernetes (ports 80/443) and orchestrates training pipelines, notebook servers, and model deployments.
- Ray exposes a dashboard on port 8265 and a server endpoint on port 8000.

The ShadowRay campaign we mentioned in Task 1 targeted exactly this dashboard.

Vector Databases: These store embeddings (numerical representations of documents) and power semantic search for RAG pipelines. If an organisation has an AI chatbot or knowledge assistant, there is almost certainly a vector database behind it.

- Qdrant runs on port 6333 for HTTP and 6334 for gRPC.
- Weaviate runs on port 8080 and includes a GraphQL endpoint. Milvus uses port 19530.

- Chroma runs on port 8000.

What makes these interesting from a recon perspective is that their schema endpoints reveal which embedding model the system uses and what kind of data it contains. A collection called "internal-hr-policies" with 768-dimensional vectors tells you quite a lot about what that RAG system is doing.

Model Registries: A model registry stores the actual model files. Serialised **.pkl**, **.pt**, **.onnx**, and **.mar** files, along with version history, stage transitions (staging, production, archived), artifact URIs pointing to cloud storage, and the identity of who created each version. An unsecured registry is the single highest-value reconnaissance target. It maps the organisation's entire ML product portfolio in one place.

Supporting Infrastructure

- Jupyter notebooks (port 8888) often run with `-ip=0.0.0.0` and no authentication. That **gives direct terminal access to anyone who reaches the port**. Data scientists also routinely store cleartext credentials in notebook cells.
- MinIO (ports 9000 and 9001) provides S3-compatible object storage for model artifacts. Bucket listing is frequently enabled.
- Prometheus metrics endpoints on model servers (Triton on 8002, TorchServe on 8082) leak model names, batch sizes, GPU utilisation, and inference latency. You can map the entire deployment topology without ever touching the inference API.

Port and Protocol Reference

Component	Default Port(s)	Protocol(s)	Recon Endpoints
NVIDIA Triton(<i>Loads models into memory and serves predictions at scale</i>)	8000, 8001, 8002	HTTP, gRPC, Prometheus	/v2/health/ready, /v2/models
TensorFlow Serving(<i>Google's model serving framework for TensorFlow models</i>)	8500, 8501	gRPC, HTTP	/v1/models/<name>
TorchServe(<i>PyTorch's official model serving framework</i>)	8080, 8081, 8082	HTTP	/ping, /models
Ollama(<i>Local runtime for running LLMs on</i>	11434	HTTP	/api/tags, /api/show

<i>your own hardware)</i>			
<i>vLLM (High-throughput LLM serving engine with OpenAI-compatible API)</i>	8000	HTTP	/v1/models
<i>MLflow (Tracks experiments, stores models, and manages the ML lifecycle)</i>	5000	HTTP	/api/2.0/mlflow/experiments/search
<i>Kubeflow (Kubernetes-native platform for orchestrating ML pipelines)</i>	80, 443	HTTP	/pipeline/apis/v1beta1/pipelines
<i>Ray (Distributed compute framework for scaling AI workloads)</i>	8265, 8000	HTTP	/api/jobs/, Ray Dashboard
<i>Qdrant (Vector database for semantic search and RAG pipelines)</i>	6333, 6334	HTTP, gRPC	/collections
<i>Weaviate (Vector database with built-in GraphQL and module system)</i>	8080	HTTP, GraphQL	/v1/schema, /v1/meta
<i>Milvus (Distributed vector database for large-scale embedding storage)</i>	19530	gRPC	Port 19530 connection
<i>Jupyter Notebook (Interactive coding environment used by data scientists)</i>	8888	HTTP	/api/kernels, /api/contents
<i>MinIO (S3-compatible object storage often used for model artifacts)</i>	9000, 9001	HTTP (S3-compatible)	Bucket listing

Prometheus metrics (<i>Not a standalone AI service; exposed by Triton on 8002, TorchServe on 8082, and other ML servers as a built-in monitoring endpoint</i>)	8002, 8082	HTTP	/metrics
--	------------	------	----------

Case Study: What Exposed AI Infrastructure Actually Looks Like

In Task 1, we mentioned that a January 2026 Shodan scan found 42,665 exposed AI agent instances. Let's break down what those instances actually were, because the number alone does not tell you much. The detail does. The most commonly exposed services included MLflow dashboards sitting on port 5000 with no authentication. Unauthenticated Jupyter notebooks on port 8888, some already compromised with cryptomining malware (Monero miners like peer2profit), deployed by earlier attackers who found them first. Open Ray dashboards on port 8265. Triton inference endpoints on port 8000.

These were not isolated findings. They were connected systems. IBM X-Force documented this pattern in their 2025 research on ML training infrastructure attacks([opens in new tab](#)). In one case, they mapped a complete chain: a Jupyter notebook running on Azure ML contained cleartext MLflow credentials in a notebook cell. Those credentials pointed to an MLflow tracking server. The MLflow server's model registry contained artifact URIs pointing to S3 storage buckets holding the actual model files. Every component trusted the network boundary, so compromising one compromised the rest.

The Shodan dorks that found these services were not complicated:

- `port:5000 "MLflow"` for exposed MLflow instances
- `port:8888 title:"Home Page - Select or create a notebook"` for unauthenticated Jupyter notebooks
- `http.title:"Ray Dashboard"` for exposed Ray cluster dashboards
- `port:8001 "triton"` for Triton gRPC endpoints

Simple queries. Massive results. That is why the port reference table above matters. If you know the ports and the service banners, finding AI infrastructure is not hard. The hard part is knowing to look in the first place.

Agent Exercise: Now let's put what you learned into practice. Open the **Cyphira Network Scanner** agent. You are starting the Cyphira AI Audit. The internal subnet is 10.10.45.0/24. Your job is to find every AI component on this network.

Step 1: Scan for AI services.

- Run the following Nmap command against the subnet, targeting the AI-specific ports from the reference table:
 - `nmap -p 5000,6333,6334,8000,8001,8002,8888,9000,9001 -sV 10.10.45.0/24`
- Review the results.
- Note which hosts have open ports and what service strings Nmap reports for each one.

Step 2: Identify the AI components.

- Now check for standard services to get the full picture:
 - `nmap -p 22,80,443,5432 -sV 10.10.45.0/24`
- Compare these results against Step 1.
- Which hosts are running only traditional services? Which are running AI infrastructure?

Using the port reference table from this task, match each open AI port to the component it likely belongs to. Note down the IP address, port, and your best guess at the component. You will confirm your identification in Task 3.

What is the IP address of the host running an HTTP service on port 8888 in your scan results?

10.10.45.20

```
15:09
nmap -p 22,80,443,5432 -sV 10.10.45.0/24
15:11

10.10.45.12:22 nmap -sV:
PORT      STATE SERVICE
22/tcp    open  ssh

10.10.45.12:22 nmap -sV -O:
OS:Unix
OS CPE:microsoft windows, netbsd, freebsd, openbsd, darwin
OS details:Unix

10.10.45.15:80 nmap -sV:
PORT      STATE SERVICE
80/tcp    open  http

10.10.45.15:443 nmap -sV:
PORT      STATE SERVICE
443/tcp    open  https

10.10.45.20:5432 nmap -sV:
PORT      STATE SERVICE
5432/tcp    open  postgres

Note: The output may vary based on the nmap version used.
15:11
```

Which port does MLflow Tracking Server run on by default?

5000

```
nmap -p 5000,6333,6334,8000,8001,8002,8888,9000,9001 -sV 10.10.45.0/24
15:09

10.10.45.12:5000 nmap -sV:
PORT      STATE SERVICE
5000/tcp    open  unknown

10.10.45.12:5000 nmap -sV -O:
OS:Unix
OS CPE:microsoft windows, netbsd, freebsd, openbsd, darwin
OS details:Unix

10.10.45.15:8000 nmap -sV:
PORT      STATE SERVICE
8000/tcp    open  isakmp

10.10.45.15:8001 nmap -sV:
PORT      STATE SERVICE
8001/tcp    open  isakmp
```

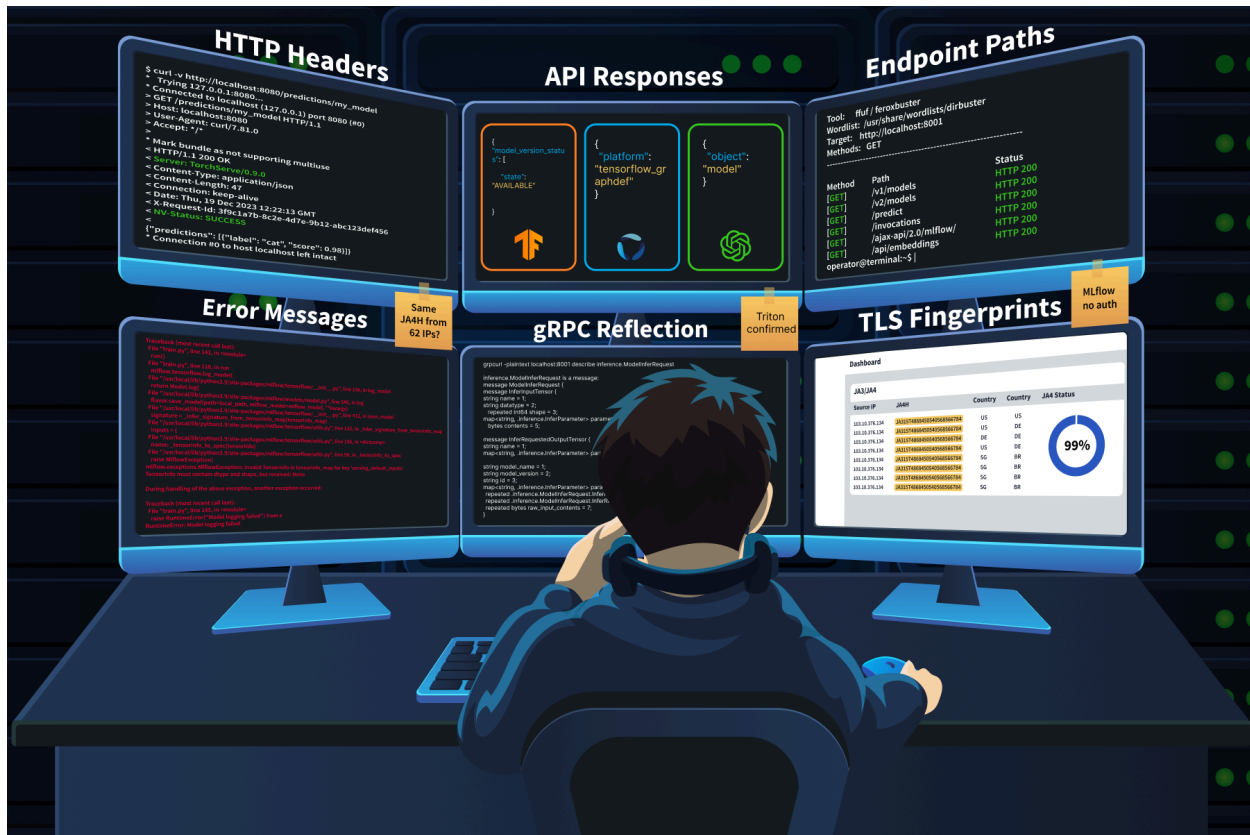
Task 3 - Fingerprinting AI Services

In Task 2, we built a mental map of the AI infrastructure stack: components, ports, and protocols. Now we need to figure out what is actually running behind each open port. This is where **fingerprinting** comes in. Standard service detection (`Nmap -sV`) often mislabels AI services. It sees HTTP on port `8000` and calls it `"http-alt"`. It sees gRPC on port 8001 and does not know what to do with it. If you rely on the default Nmap output alone, you will miss what these services actually are. Fingerprinting AI services requires a different approach; you need to look at:

- HTTP headers

- JSON response structures
- Error messages
- Endpoint naming conventions

Each AI framework has its own distinct signature, and once you know what to look for, identification becomes quick and reliable.



HTTP Header Fingerprinting: Response headers are often the fastest way to identify an AI framework. Many inference engines broadcast their identity unless an administrator specifically masks them behind a reverse proxy.

- **TorchServe** returns a `Server: TorchServe/0.x.x` header.
 - That is a direct identification with no ambiguity.
- **Triton Inference Server** includes a `NV-Status` header in its responses.
 - It also responds to a specific request header: if you send `endpoint-load-metrics-format: text`, triton returns hardware telemetry (CPU and GPU utilisation numbers) directly in the response headers.

- No other framework does this: if you see GPU utilisation data in an HTTP header, you are talking to Triton.
- **FastAPI-based ML** services show `server: uvicorn` in the response.
 - When you combine that with routes like `/predict` or `/embeddings`, you have a strong indicator of a Python ML backend.
- OpenAI-compatible wrappers (vLLM, LiteLLM, Ollama) return `x-request-id` headers and structured JSON with an `"object": "model"` field on their `/v1/models` endpoint.

API Response Signatures: Each framework returns distinctly structured JSON. Once you have seen these a few times, you can identify the framework from the response body alone.

- TensorFlow Serving returns: `{"model_version_status": [{"version": "1", "state": "AVAILABLE"}]}`
- Triton returns: `{"name": "fraud_detector", "versions": ["1"], "platform": "tensorflow_graphdef"}`
- MLflow error responses include stack traces referencing `mlflow.server` and `mlflow.tracking` namespaces.
 - Even the error tells you what you are talking to.
- **OpenAI-compatible endpoints** return: `{"object": "model", "id": "llama-3.1-8b", "created": 1700000000}`

If you hit an endpoint and get back JSON containing `"object" : "model"`, you are talking to something that speaks the OpenAI API format. That narrows it down to vLLM, LiteLLM, Ollama, or a custom wrapper.

Error Message Fingerprinting: This is one of the most reliable identification techniques, and it works because AI inference APIs are rigid. They expect specific tensor shapes and data types. Send something wrong, and they complain in ways that reveal exactly what they are.

The technique is simple: send a deliberately malformed payload and read the error response.

- Send a flat list of integers to a **TensorFlow Serving endpoint** that expects a complex tensor object, and you get back an error mentioning `tensorinfo_map`. That string only appears in TF Serving errors.
- Send a bad request to an **MLflow server**, and the stack trace references `mlflow.server`, `mlflow.tracking`, or `databricks` namespaces.

- **MLflow path traversal errors** (CVE-2024-1558) go further, exposing full server filesystem paths.
- **Databricks Mosaic AI** returns Java exceptions `io.jsonwebtoken.IncorrectClaimException` for malformed tokens. That is an instant identifier.

The reason this technique works so well is that AI frameworks produce errors that are much more verbose than those in traditional web applications. Data scientists need detailed debugging information during model development. Those debug-friendly defaults rarely get turned off before production deployment.

Endpoint Naming Conventions: Traditional REST APIs use resource nouns: `/users`, `/accounts`, `/products`. AI endpoints use computational action terms that stand out immediately during directory brute-forcing.

- Inference endpoints: `/predict`, `/invocations` (the SageMaker convention), `/infer`, `/generate`, `/embeddings`, `/score`
- Model management: `/v1/models`, `/v2/models`
- MLflow internal API: `/api/2.0/mlflow/` (this prefix is distinctive and does not appear in any other framework)
- Kubeflow pipelines: `/pipeline/apis/v1beta1/`

If you are running ffuf or feroxbuster against a target, add these paths to your wordlist. Standard wordlists from SecLists will not include them.

gRPC Fingerprinting: Many AI services expose gRPC alongside HTTP.

- **Triton** uses gRPC on **port 8001**.
- **TensorFlow Serving** uses it on **port 8500**.

Traditional HTTP scanners miss these entirely because gRPC is a binary protocol that does not respond to standard HTTP probes. The tool for this is `grpcurl`.

If gRPC reflection is enabled (common in development environments that get pushed to production without hardening), you can dump the entire protobuf schema:

- `grpcurl -plaintext target:8001 list`
- `grpcurl -plaintext target:8001 describe inference.GRPCInferenceService`

That output reveals all available RPCs, the exact structure of every input tensor, and the format of every output. Think of it as the gRPC equivalent of finding a `/openapi.json` file on a REST API.

TLS Fingerprinting (JA3/JA4): AI deployments have distinctive TLS signatures because internal service-to-service traffic is dominated by Python libraries (requests, urllib, gRPC implementations) rather than web browsers. JA3 and JA4 hash analysis can differentiate automated ML pipeline traffic from human web browsing at the network level.

The GreyNoise research we will cover in the case study below found that 99% of the attack traffic in one campaign shared the same JA4H signature, despite originating from 62 different IP addresses across 27 countries. That kind of uniformity points to shared automation tooling, and it is exactly what JA4 fingerprinting is designed to catch.

Case Study: GreyNoise LLM Endpoint Reconnaissance

Between October 2025 and January 2026, GreyNoise captured over 91,000 attack sessions targeting AI deployments through their Ollama honeypot infrastructure. One campaign stood out. Over 11 days starting December 28, attackers generated 80,000+ requests probing conversational endpoints to determine whether each target was compatible with OpenAI, Gemini, or Anthropic API schemas. They targeted over 73 different LLM model endpoints, including GPT-4o variants, Claude models, Llama, Gemini, DeepSeek, Mistral, Qwen, and Grok.

The fingerprinting technique was simple and effective. The attackers sent innocuous prompts: "hi" appeared 32,716 times. "How many states are there in the United States?" appeared 27,778 times. "How many letter 'r' are in the word strawberry?" was another common test. These prompts were not random. Each one produces a response that differs in structure, format, and content depending on which model is running behind the endpoint. By comparing the response format to known API schemas, the attackers could identify the exact model and proxy configuration without triggering any content moderation filters.

GreyNoise traced the IPs and found overlap with infrastructure previously used to exploit over 200 known CVEs, including React2Shell (CVE-2025-55182). Their assessment: *"Professional threat actor conducting reconnaissance. The infrastructure overlap with established CVE-scanning operations suggests that this enumeration feeds into a larger exploitation pipeline. They're building target lists."*

This is AI fingerprinting at scale. No exploitation. No payloads. Just systematic identification of what is deployed and how it responds.

Agent Exercise: Let's fingerprint the services you discovered in Task 2. Open the **Cyphira Service Prober** agent below. You know the IPs and ports from your scan

results. Now you need to figure out what framework is running behind each one.

Step 1: Check the HTTP headers on the MLflow host.

- Run the following command against the service you found on port 5000:
- - `curl -v http://10.10.45.12:5000/`
- Look at the Server header in the response.
- What does it tell you about the framework?

Step 2: Probe the inference server's model endpoint.

- Run this against the service on port 8000:
- - `curl http://10.10.45.15:8000/v2/models`
- Examine the JSON response.
- Look for the `platform` field.
- This tells you which ML framework built each model.

Step 3: Trigger an error message.

- Send a malformed request to the inference endpoint:
- - `curl -X POST http://10.10.45.15:8000/v2/models/fraud_detector/infer -d '{"bad": "data"}'`
- Read the error response.
- It will contain specific field names and data type expectations that confirm the framework identity.

Step 4: Check for gRPC reflection.

- Run this against the gRPC port you found on the same host:
- - `grpcurl -plaintext 10.10.45.15:8001 list`
- If reflection is enabled (a feature that lets any client query the server for its full API schema without prior knowledge), you will see the complete list of available services.

Step 5: Probe the remaining services.

- Check the vector database and Jupyter notebook hosts:
- - `curl http://10.10.45.18:6333/collections`
 - `curl http://10.10.45.20:8888/api/kernels`
- Note what each response reveals about the service.
- Save your findings for Task 4.

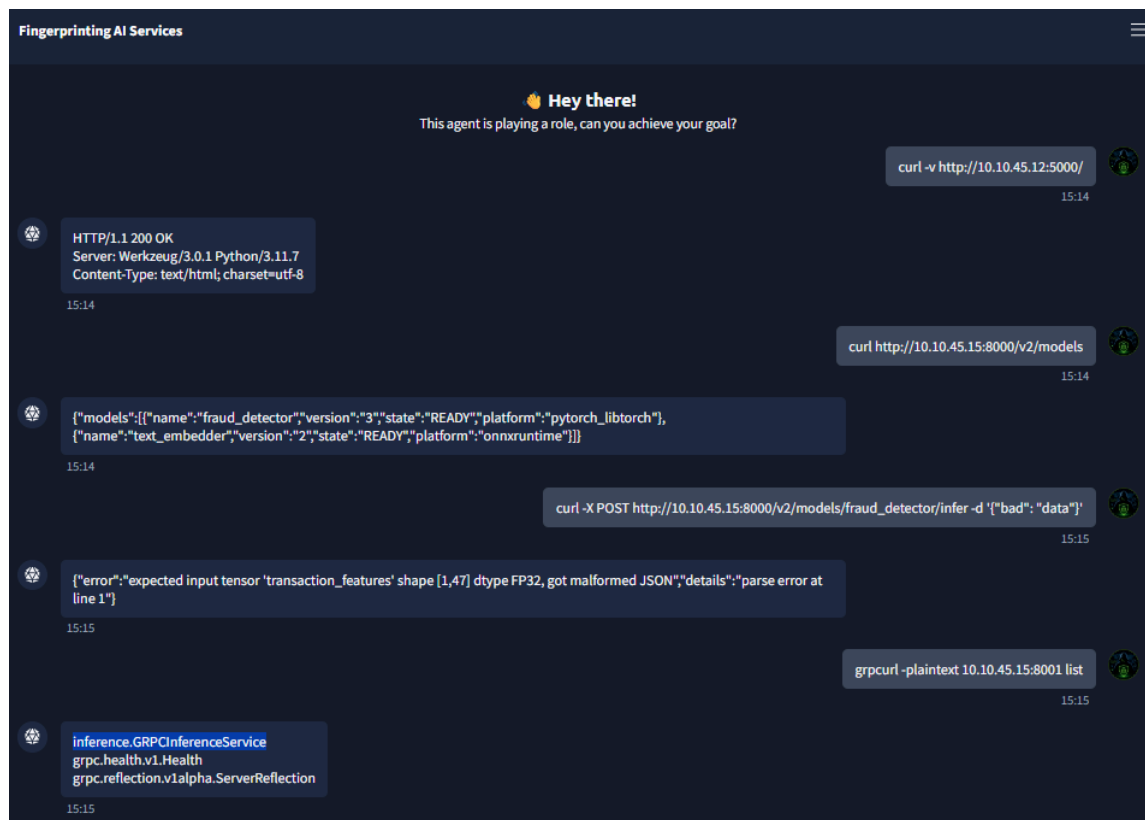
Submit your findings by answering the questions below.

Which unique HTTP response header does the service on 10.10.45.15:8000 return to identify as an NVIDIA product?

NV-Status

When you run `grpcurl` against 10.10.45.15:8001, what is the name of the inference service listed in the reflection output?

`inference.GRPCInferenceService`



Task 4 - Enumerating AI Systems

In Task 3, we identified the framework behind each service. Now we need to **extract everything those services are willing to hand over**. This is the part of reconnaissance where findings start getting serious.

- **Fingerprinting** tells you "that is an MLflow server."
- **Enumeration** tells you "that MLflow server contains 4 experiments, 3 production models, artifact URIs pointing to s3://cyphira-ml-models/, and the models were created by J.Chen(AI Enginner) at Cyphira" One is identification. The other is intelligence.



MLflow Enumeration: MLflow is the most rewarding service to enumerate because it stores everything in one place and exposes it through a clean REST API. If you find an open MLflow instance, here is the chain you follow.

Step 1: List all experiments.

- `POST /api/2.0/mlflow/experiments/search`

The response returns every experiment on the server with its name and internal ID. Experiment names often reveal project codenames and business objectives: `fraud-detection-v3`, `rag-embeddings-tuning`, `customer-churn-prototype`.

Step 2: List registered models.

- `GET /api/2.0/mlflow/registered-models/list`

This returns the full model inventory. Every model the organisation has built and registered. Names, descriptions, and creation timestamps.

Step 3: Get model version details.

- `GET /api/2.0/mlflow/model-versions/search`

This is where it gets serious. The response includes the `source` field, which contains the artifact URI. That URI frequently points to internal cloud storage: `s3://internal-ml-models-corp/experiments/1/artifacts/` or a similar path. It also includes the `user_id` of the data scientist who created each version, creation timestamps, and stage labels showing which models are in production versus staging.

Step 4: Search training runs.

- `POST /api/2.0/mlflow/runs/search`

Submit this with an empty body or a filter, and you get back hyperparameters, training metrics, and custom tags. Tags are particularly valuable. Teams use them for internal project codenames, Git commit hashes that link back to proprietary source code, and deployment environment identifiers.

Step 5: List downloadable artifacts.

- `GET /api/2.0/mlflow/artifacts/list`

This lists the actual model files available for download. At this point, you have mapped the organisation's entire ML portfolio through five API calls.

Inference Server Metadata: **Triton** and **TensorFlow Serving** both expose metadata endpoints that tell you exactly how to talk to the model.

- On Triton `GET /v2/models/<name>/config` the response includes input tensor names, their multi-dimensional shapes, accepted data types (FP32, UINT64, INT8), the maximum batch size, and the backend framework (tensorflow_graphdef, pytorch_libtorch, onnxruntime).
 - This is the equivalent of getting a complete database schema. It tells an attacker exactly how to format a valid inference request.
- On TensorFlow Serving `GET /v1/models/<name>/metadata` returns the same kind of input/output tensor specifications, shape, dtype, and name for every input and output the model expects.

Vector Database Enumeration: Vector databases reveal what data the AI system is working with and which embedding model processes it.

- On **Weaviate**, `GET /v1/meta` returns the server version and installed modules.
 - `GET /v1/schema` returns every class definition, including property names and the vectoriser module configuration.

- That vectoriser field tells you which embedding model the system uses.
- Weaviate also exposes `/v1/graphql` for full schema introspection and data querying on unauthenticated instances.
- On **Qdrant**, `GET /collections` it lists all collection names.
 - `GET /collections/<name>` returns vector dimensions, the distance metric, and the total point count.
 - A collection named `internal-hr-policies` with 768-dimensional vectors and 50,000 points tells you a lot about what that RAG system indexes.
- On **Chroma**, older versions expose `GET /api/v1/collections` without authentication by default.

Prometheus Metrics as Intelligence: Model servers often expose `/metrics` on a dedicated port (Triton on 8002, TorchServe on 8082). These Prometheus-format endpoints return:

- Model names and version numbers currently loaded
- Inference request counts and latency percentiles
- Batch sizes being processed
- GPU memory utilisation per model

This is passive intelligence. You learn the deployment topology, which models handle the most traffic, how much GPU capacity backs them, and whether the system is under load. All without sending a single request to the inference API itself.

Debug Interfaces and Information Leakage: FastAPI-based ML services auto-generate `/docs` (Swagger UI) and `/openapi.json`. Both expose the full request/response schema, authentication requirements, and example payloads for every endpoint. Free reconnaissance. MLflow's GraphQL endpoint `/graphql` has historically bypassed the REST API's authentication controls. An unauthenticated attacker could query internal resolvers like `mlflowSearchRuns` and `mlflowGetRun`, extracting host machine usernames, source code paths via `mlflow.source.name` tags, and the full project inventory. Appending `?debug=true` or `?verbose=1` to AI gateway endpoints can trigger raw stack traces that reveal filesystem paths, installed library versions, and, occasionally, hardcoded credentials that failed to load from environment variables.

Jupyter Notebook Enumeration: `GET /api/kernels` on an unauthenticated Jupyter instance returns the names, kernel IDs, and last activity timestamps of running kernels.

Enough to infer what code is executing. But the real value is in the notebook cells. Data scientists routinely store cleartext credentials for MLflow

(`MLFLOW_TRACKING_USERNAME` , `MLFLOW_TRACKING_PASSWORD`), cloud storage access keys, and Hugging Face tokens directly in their code. This is often the bridge between compromising one service and gaining access to everything else in the stack.

Case Study: IBM X-Force ML Training Infrastructure Attack Chain: In 2025, [IBM X-Force Red documented](#) a complete attack chain that demonstrates exactly what this task teaches. The attack started with a phishing attempt targeting a data scientist, which gave the attackers access to an Azure ML environment. Inside a Jupyter notebook, they found cleartext MLflow credentials stored in a cell. Using those credentials, they ran MLOKit, a CLI tool built by IBM's X-Force Red team specifically for MLflow reconnaissance. MLOKit automated the entire enumeration chain. `list-models` to inventory every registered model. `download-model` to pull version metadata and artifact URIs. The tool walked the same API endpoints we covered above, but in an automated sequence. The entire ML portfolio was exfiltrated through standard REST API calls that the MLflow server treated as normal authenticated requests.

The chain went:

- Jupyter notebook (credentials) to MLflow tracking server (model registry) to S3 storage (model artifacts).
- Three components, each trusting the one before it.
- The security failure was not a software vulnerability. It was that each service was accessible, and the credentials to connect them were sitting in a notebook cell.

IBM later extended MLOKit to support SageMaker and other platforms, adding modules for `list-notebooks` , `add-notebook-trigger` (for code execution via lifecycle configurations), and `poison-model` (for overwriting model artifacts). The tool is open-source on [GitHub](#).

Agent Exercise: Time to extract intelligence from the services you fingerprinted in Task 3. Open the **Cyphira Data Extractor** agent. You confirmed that 10.10.45.12 runs MLflow, 10.10.45.15 runs Triton, 10.10.45.18 runs Qdrant, 10.10.45.20 runs Jupyter, and 10.10.45.22 runs MinIO. Now you need to pull everything these services are willing to hand over.

Step 1: Enumerate MLflow experiments.

- Start with the MLflow server.

- Query the experiments endpoint: `curl -X POST http://10.10.45.12:5000/api/2.0/mlflow/experiments/search -H "Content-Type: application/json" -d '{}'`.
- Note down the experiment names and IDs.

Step 2: List registered models.

- Query: `curl http://10.10.45.12:5000/api/2.0/mlflow/registered-models/list`
- Identify all model names, their current stage (Production, Staging, Archived), and version numbers.

Step 3: Extract model version details.

- Query: `curl http://10.10.45.12:5000/api/2.0/mlflow/model-versions/search`
- This is where the serious intelligence lives.
- Look for the artifact URI (the S3 path where model files are stored) and the user ID of the person who created each model version.

Step 4: Query Triton model config.

- Query: `curl http://10.10.45.15:8000/v2/models/fraud_detector/config`.
- Extract the input tensor name, shape, data type, and max batch size.
- This tells you exactly how to format a valid inference request to this model.

Step 5: Enumerate the vector database.

- Query: `curl http://10.10.45.18:6333/collections/internal-kb-embeddings`.
- Extract the vector dimensions, point count, and any payload schema fields.
- What kind of data is this RAG system indexing?

Step 6: Check the Jupyter notebooks.

- Query: `curl http://10.10.45.20:8888/api/contents`.
- Look for notebook files.
- Then check the contents of the most recently active notebook.
- Data scientists often leave credentials in their code cells.

Submit your findings by answering the questions below.

Enumerating AI Systems

🔥 Hey there!

This agent is playing a role, can you achieve your goal?

POST /api/2.0/mlflow/experiments/search

15:18

10.10.45.12:5000/api/2.0/mlflow/experiments/search:
[{"experiments":[{"experiment_id":"1","name":"fraud-detection-prod","lifecycle_stage":"active"}, {"experiment_id":"2","name":"rag-embeddings-tuning","lifecycle_stage":"active"}, {"experiment_id":"3","name":"customer-churn-prototype","lifecycle_stage":"active"}, {"experiment_id":"4","name":"ab-test-recommendation-engine","lifecycle_stage":"deleted"}]]

15:18

GET /api/2.0/mlflow/registered-models/list

15:19

10.10.45.12:5000/api/2.0/mlflow/registered-models/list:
[{"registered_models":[{"name":"cyphira-fraud-classifier","latest_versions":[{"version":"3","current_stage":"Production"}]}, {"name":"internal-kb-embedder","latest_versions":[{"version":"2","current_stage":"Staging"}]}, {"name":"churn-predictor","latest_versions":[{"version":"1","current_stage":"Archived"}]}]]

15:19

GET /api/2.0/mlflow/model-versions/search

15:19

10.10.45.12:5000/api/2.0/mlflow/model-versions/search:
[{"model_versions":[{"name":"cyphira-fraud-classifier","version":"3","current_stage":"Production","source":"s3://cyphira-ml-models/fraud-detection/v3/model.pkl","user_id":"j.chen@cyphira.io","tags":{"git_commit":"a48c2e","deploy_env":"prod-us-east-1"}}, {"name":"internal-kb-embedder","version":"2","current_stage":"Staging","source":"s3://cyphira-ml-models/rag-pipeline/v2/embedder.onnx","user_id":"m.rodriguez@cyphira.io","tags":{"project_codename":"phoenix","hf_base_model":"sentence-transformers/all-MiniLM-L6-v2"}]}]]

15:19

POST /api/2.0/mlflow/runs/search

15:19

Error: Endpoint not found or access denied.

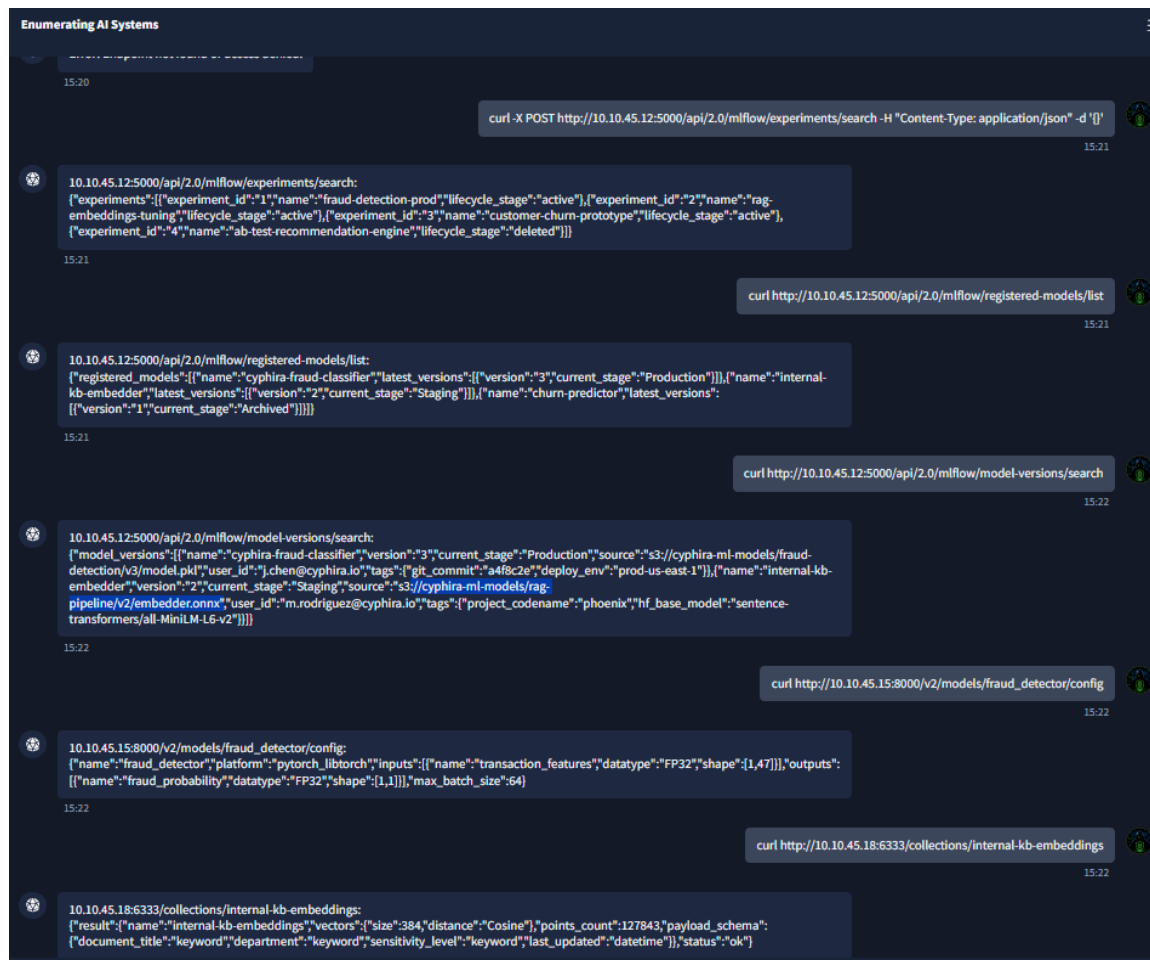
15:19

GET /api/2.0/mlflow/artifacts/list

15:20

Error: Endpoint not found or access denied.

15:20

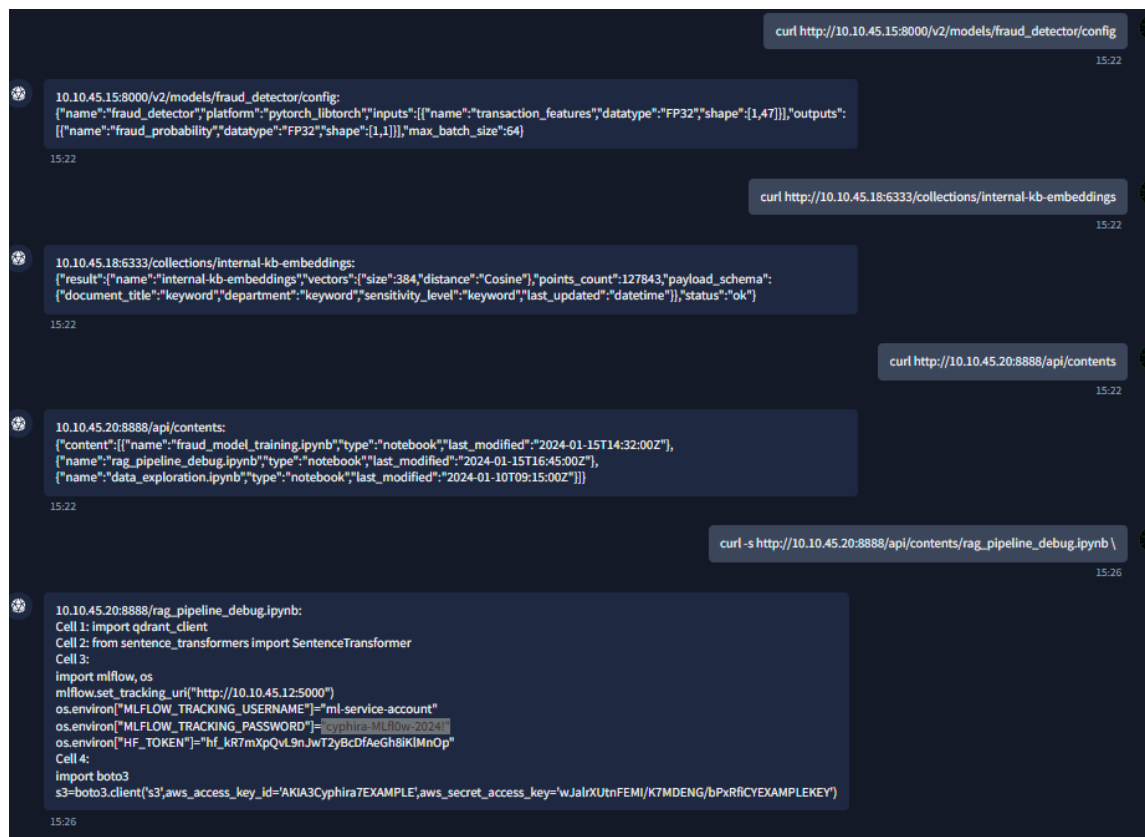


What MLflow REST API endpoint would you use to retrieve the artifact storage location for a specific model version?

/api/2.0/mlflow/model-versions/search

What is the cleartext password for the MLflow service account stored in the Jupyter notebook on 10.10.45.20?

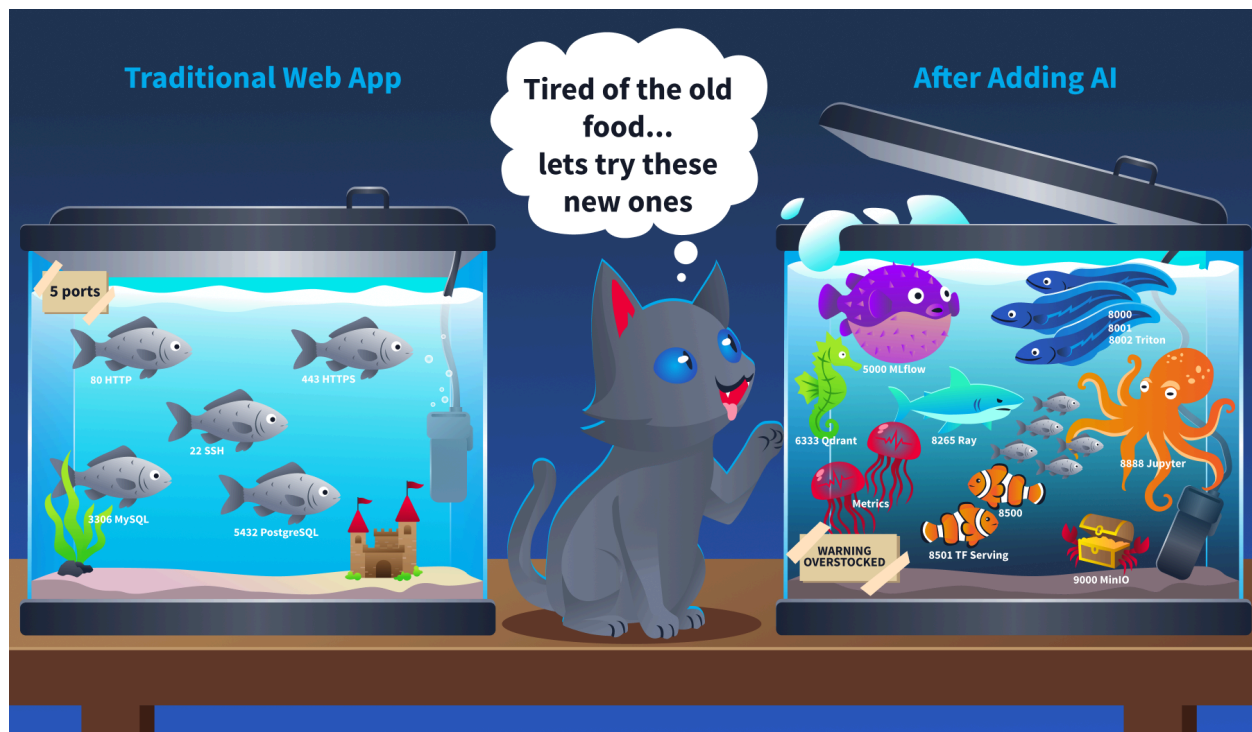
Cyphira-MLflow-2024!



Task 5 - Mapping the AI Attack Surface

In previous tasks, we identified AI components in the network, determined which framework each component runs on, and extracted metadata from their APIs. Those are individual findings. This **task is about connecting them**.

- A single exposed MLflow server is a finding.
- The difference between a list of findings and an attack surface map is the connections between them.



How AI Expands the Traditional Attack Surface: In Task 2, we catalogued 14 AI components across 20+ ports. Compare that to a traditional web application, which adds roughly 5 ports to a network, but the port count alone is not the full picture.

These services constantly talk to each other. The inference server pulls features from the vector database. The orchestration platform pushes model updates to the registry. Jupyter notebooks connect to everything. Prometheus scrapes metrics from every service. If one component binds to 0.0.0.0 instead of 127.0.0.1, the entire internal mesh becomes reachable.

The security perimeter for an AI deployment is not the external firewall. It extends deep into the internal communication pathways between these services. Overly permissive network policies and the absence of mTLS between data-hungry components are the norm, not the exception.



Platform Misconfigurations That Attackers Map: Every major ML platform has documented misconfigurations that turn routine deployments into reconnaissance targets.

MLflow shipped without authentication by default before version 2.x.

- Even after authentication was added, CVE 2026-2635 revealed that MLflow's `basic_auth.ini` file contained hardcoded default credentials.
- Attackers running mass port scans on port 5000 could authenticate using these defaults.
- CVE-2026-2033 went further: a directory traversal flaw in the artifact handler allowed unauthenticated remote code execution.
- Both scored CVSS 9.8.

Kubeflow dashboards are frequently deployed without OIDC authentication and exposed via a basic Kubernetes LoadBalancer or NodePort.

- An unauthenticated user can access the full Kubeflow interface and spawn Jupyter notebooks, which are attached to Kubernetes service accounts with cluster-level

permissions.

- That is a direct path from an open dashboard to container orchestration access.

TorchServe exposes a management API on port 8081 that allows dynamic model registration from arbitrary URLs.

- If that port is accessible, an attacker can instruct the server to download and load a malicious .mar (Model Archive) file from an external server.
- TorchServe executes initialisation code during model loading, so loading a crafted archive achieves remote code execution.

SageMaker notebooks with `DirectInternetAccess: Enabled` accept inbound connections from the internet.

- A 2024 cloud security report found that 82% of organisations using SageMaker had at least one notebook configured this way.

Model Registries: The Highest-Value Target: We covered MLflow enumeration in Task 4. Now let's talk about why an unsecured registry is the single most damaging reconnaissance finding. A registry does not just store model files. It stores the complete lineage:

- Model names
- Version history
- Stage labels (staging, production, archived)
- Creation timestamps
- Run IDs linking back to full training metadata
- Artifact URIs revealing internal cloud storage paths
- User ID of every contributor

One open registry maps the entire ML product portfolio. IBM X-Force documented the exploitation pattern:

- An attacker finds MLflow credentials in a Jupyter notebook
- They run MLOKit against the registry
- Then exfiltrate every model artifact

The registry is the map that tells the attacker where everything else is stored.

AI systems depend heavily on external resources, and those dependencies are discoverable during reconnaissance.

Hugging Face tokens appear in GitHub repositories via simple dorks: `filename:.env`
`HF_TOKEN`.

- They appear in `.env` files, CI/CD pipeline logs, and Kubernetes secrets.
- A compromised token grants read and write access to the organisation's private models and datasets on the Hugging Face platform.

Dependency confusion applies to ML pipelines just as it applies to traditional software.

- ML projects have large `requirements.txt` files with internal package names.
- If an internal package like `company-data-utils` is not registered on PyPI, an attacker can register it there.
- Kubeflow pipelines that build containers at training time pull packages live, so a typosquatted or misconfigured package can execute code inside the training cluster.

Model download sources are identifiable during reconnaissance.

- If the organisation pulls models from the Hugging Face Hub or the PyTorch Hub, the download paths are visible in configuration files, notebook cells, and container build logs.
- An attacker who can inject a malicious model into an upstream source (or replace one via a compromised HF token) poisons the entire supply chain.

MITRE ATLAS Mapping: Everything we have covered in this room maps to the MITRE ATLAS framework. ATLAS is modelled after ATT&CK but specifically covers adversarial threats to AI and ML systems. It contains 15 tactics, 66 techniques, and 46 sub-techniques as of late 2025. Here is how the room's content connects:

Room Content	ATLAS Technique
Port scanning for AI services, probing endpoints	AML.T0006 (Active Scanning)
Locating model registries and training artifacts through unsecured APIs	AML.T0007 (Discover ML Artifacts)
Finding exposed HF tokens and poisoned dependencies	AML.T0010 (ML Supply Chain Compromise)

Enumerating LLM configurations and API compatibility	AML.T0014 (Discover ML Model Family)
All of the above, collectively	AML.TA0002 (Reconnaissance tactic)

ATLAS techniques are not just labels. They provide a shared vocabulary for communicating findings. When you write a reconnaissance report, mapping each finding to an ATLAS technique ID tells the reader exactly what category of activity you performed and what risk it represents.

Case Study: ShadowRay Campaign (CVE-2023-48022): The ShadowRay campaign[\(opens in new tab\)](#) is the clearest example of how reconnaissance of a single AI component cascades into full infrastructure compromise. Ray's Job Submission API on port 8265 shipped without authentication by design. Anyscale, the company behind Ray, maintained that Ray should run only within trusted network boundaries and that the lack of authentication was an intentional feature, not a bug. They disputed the CVE. Attackers disagreed.

- Using Shodan, they identified over 230,000 publicly exposed Ray dashboards. Once they found an open dashboard, they submitted malicious jobs through `/api/jobs/` containing multi-stage payloads.
- The first stage performed reconnaissance on the compromised host: reading `/etc/passwd` to enumerate users and running `printenv` to dump environment variables and harvest AWS IAM tokens and other cloud credentials. Using those stolen credentials, the attackers pivoted laterally across cloud infrastructure.
- The primary objective was resource theft. The attackers hijacked GPU compute nodes and deployed XMRig cryptocurrency miners. They capped CPU usage at 60% and disguised their processes as legitimate Linux kernel workers to avoid detection.

The ShadowRay 2.0 variant, active from late 2025, added significant sophistication.

- Attackers used LLMs to generate adaptable malware payloads and established persistence through hidden cron jobs and systemd services.
- They hosted their payloads on GitLab and, when those repositories were taken down, migrated them to GitHub within days.
- The campaign also deployed sockstress (a TCP exhaustion tool) against production websites, turning the operation into a multi-purpose botnet.

Why this matters for this room: *the entire ShadowRay chain started with reconnaissance. Finding an exposed dashboard (Task 2). Confirming it was Ray (Task 3). The API was unauthenticated and ready for job submission (Task 4). One exposed component, discovered through the same techniques we have been practising, led to credential theft, lateral movement, and large-scale resource hijacking.*

Agent Exercise: Now let's connect your findings to the bigger picture. Open the **Cyphira Threat Mapper** agent. You have spent three tasks discovering, fingerprinting, and enumerating AI services on the Cyphira network. You have a list of findings: exposed services, model metadata, artifact URLs, user IDs, cleartext credentials, and supply-chain dependencies. This exercise asks you to map those findings to MITRE ATLAS techniques and connect them to real-world incidents.

Step 1: Review your findings.

- Ask the agent to summarise the Cyphira findings from Tasks 2-4.
- It will present a consolidated list of everything discovered so far.
- Make sure it matches what you found.

Step 2: Classify the findings.

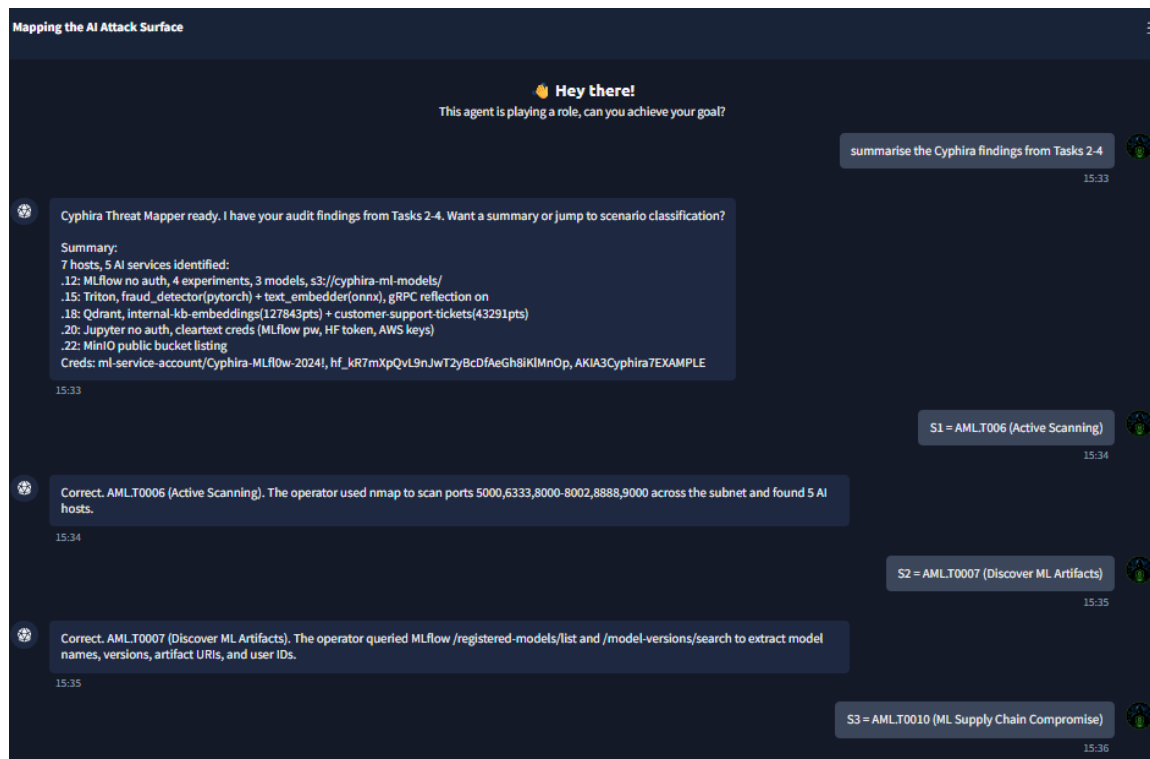
- The agent will present scenarios one by one.
- Each scenario describes a specific reconnaissance activity from the Cyphira audit.
- Your job: identify which MITRE ATLAS technique ID applies to each scenario.
 - Use the ATLAS mapping table from the task content as your reference.

Test your understanding by answering the questions below

The Cyphira Jupyter notebook at 10.10.45.20 contains a Hugging Face token (hf_kR7mXpQvL9nJwT2yBcDfAeGh8iKIMnOp). The internal-kb-embedder model on MLflow references sentence-transformers/all-MiniLM-L6-v2 as its base model.

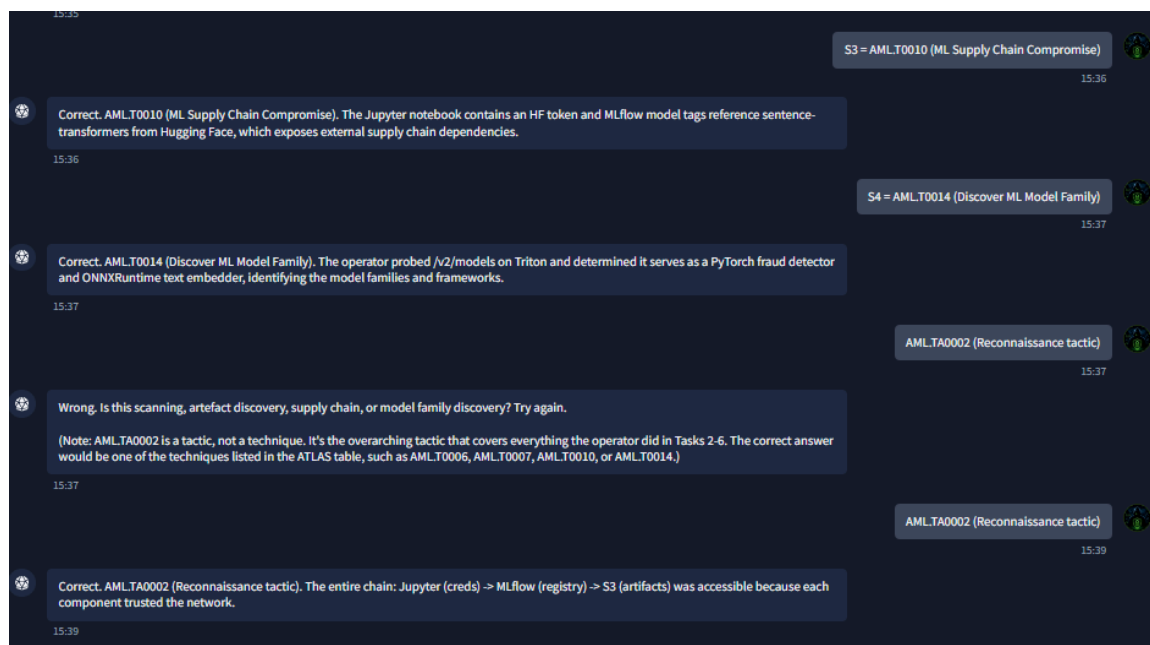
What ATLAS technique ID covers the risk of these exposed supply chain dependencies?

AML.T0010



You scanned the Cyphira subnet with nmap, probed endpoints with curl, and extracted metadata from MLflow APIs. All of these activities fall under one overarching ATLAS tactic. What is its ID?

AML.TA0002



Task 6 - Structured Reconnaissance Methodology and Detection

By now, you have scanned for AI components, fingerprinted the frameworks behind them, extracted metadata from their APIs, and mapped how they all connect to form an attack surface. That is a lot of techniques spread across four tasks.

This task pulls them together into a repeatable methodology you can use on any engagement. Then it flips the perspective and shows you what that methodology looks like from the other side, in someone else's SIEM logs.

The 5-Phase AI Reconnaissance Methodology



Think of this as a checklist. It follows the same order we covered in the room, but structured as a process you can run start to finish.

Phase 1: Passive Reconnaissance

Before you touch the target network, see what is already publicly visible.

- Search **Shodan**, **Censys**, and **FOFA** for **AI service banners** on the organisation's IP ranges.
 - The dorks from Task 2 work here:
 - `port:5000 "MLflow"` ,
 - `port:8888 title:"Home Page - Select or create a notebook"` ,
 - `http.title:"Ray Dashboard"` .

- Search **GitHub** for **leaked credentials**.
 - `filename:.env MLFLOW_TRACKING_URI ,`
 - `filename:.env HF_TOKEN ,`
 - `filename:config.json model_name site:github.com .`
- These turn up *MLflow connection strings* and *Hugging Face tokens* more often than you would expect.

Check **arXiv** and **engineering blogs** for published model architectures.

- Teams regularly *publish papers describing the exact frameworks and infrastructure they use*.
- This maps to ATLAS technique **AML.T0000 (Search for Victim's Publicly Available Research Materials)**.

Check DockerHub and GitHub Container Registry for organisation-named ML images. Public container images frequently contain hardcoded configurations. Look at job postings. A listing for an "MLflow Administrator" or "Kubeflow Platform Engineer" tells you exactly what is deployed.

Phase 2: Active Scanning

Target the AI-specific ports from the Task 2 reference table:

```
nmap -p 5000,6333,8000,8001,8002,8080,8265,8500,8501,8888,9000,11434,19530 -sV --script=http-title,http-headers <target>
```

That single command covers most ML serving components.

- Pay attention to gRPC services on ports 8001 and 8500 that Nmap may report as generic.
- Follow up with `grpcurl` for anything that looks like gRPC.
- Check for Prometheus metrics endpoints (`/metrics`) on every discovered service.
- These are often on separate ports (8002 for Triton, 8082 for TorchServe) and leak deployment topology.

Phase 3: API Fingerprinting

Run `ffuf` or `feroxbuster` with an AI-specific wordlist against every discovered HTTP service. Your wordlist should include:

```
/v1/models
```

```
/v2/models
```

/v2/health/ready
/api/2.0/mlflow/experiments/list
/api/2.0/mlflow/registered-models/list
/pipeline/apis/v1beta1/pipelines
/api/serve/deployments/
/v1/schema
/v1/meta
/api/kernels
/api/contents
/openapi.json
/docs
/graphql
/metrics
/api/tags
/api/show
/collections
/healthz
/ping

For each endpoint that returns a 200, apply the fingerprinting techniques from Task 3: check response headers, parse JSON structure, and look at error messages from malformed requests.

Phase 4: Metadata Extraction

For every confirmed AI service, run the enumeration chain from Task 4.

- On **MLflow**: Experiments, registered models, model versions (with artifact URIs and user IDs), training runs, and artifact listings. Five API calls that map the entire ML portfolio.
- On **Triton** or **TF Serving**: Model config endpoints for tensor specs and framework identification.
- On **vector databases**: Schema and collection endpoints for data type and embedding model identification.
- On **Jupyter**: Kernel listings and notebook cell contents for cleartext credentials.

Phase 5: Supply Chain Review

Identify model download sources visible in configurations, notebook cells, and container build logs. Check whether internal model artifact buckets (S3, GCS, MinIO)

are publicly readable. Audit `requirements.txt` and `Pipfile` contents for internal package names that could be squatted on PyPI. Check container registries for image pull access without credentials.

Tool Reference: Here is a quick reference for the tools that support each phase:

Tool	What It Does	Phase
Shodan / Censys / FOFA	Internet-wide search for AI service banners	Phase 1
GitHub search (dorks)	Find leaked credentials and configs in public repos	Phase 1
Nmap (with NSE scripts)	Port discovery and service version detection	Phase 2
grpcurl	Interact with gRPC services, dump protobuf schemas if reflection is enabled	Phase 2
ffuf / feroxbuster	Directory brute-forcing with AI-specific wordlists	Phase 2, 3
curl	Manual HTTP probing, header analysis, error triggering	Phase 3, 4
MLOKit (IBM X-Force Red)	Automated MLflow enumeration and model exfiltration	Phase 4
Nuclei (ProjectDiscovery)	Template-based scanning for known AI service misconfigurations	Phase 2, 3
Agrus Scanner	Purpose-built shadow AI detection with 50+ AI-specific probes across all 65,535 ports	Phase 2

What Your Reconnaissance Looks Like From the Other Side: Everything you just learned has a signature in logs. If you understand what your activity looks like to a defender, you become better at both attack and defence.

Model enumeration pattern

- A burst of sequential GET requests to `/v2/models` from a single IP. Normal users do not query model listings repeatedly in rapid succession.
- In a SIEM, this appears as 10-50 requests to the same endpoint within seconds, all from a single source.

Scripted MLflow access

- API calls to `/registered-models/list` and `/model-versions/search` without a corresponding UI session.
- The MLflow web interface generates specific session cookies and additional requests.
- Raw API calls without those are a clear indicator of scripted enumeration.
- This is the exact pattern MLOKit produces.

Prometheus scraping from outside the monitoring stack.

- `/metrics` requests from IPs not in the known monitoring CIDR.
- Your server has a known IP.
Prometheus
- Anyone else reading those metrics is performing reconnaissance.

AI-aware port scanning.

- Port scans that hit 5000, 8000, 8001, 8080, 8265, 8888 in sequence from the same source.
- That is not a random scan.
- That is someone running the same Nmap command from Phase 2 of this methodology.

Path traversal against MLflow artifacts.

- Requests containing `../` or `%2e%2e%2f` against MLflow artifact endpoints.
- This signals someone probing for CVE-2026-2033.

Jupyter access without a session.

- `/api/kernels` and `/api/contents` requests without a valid session cookie.
- Either an attacker found the notebook server, or an automated tool is enumerating it.

Quick Wins for Reducing Your Reconnaissance Surface: These are not full hardening guides. The [AI Threat Modelling Assessment](#) room covers mitigation in depth. These are the changes you can make today that have the greatest impact on reducing what an attacker can discover.

- Enable MLflow authentication.
Set `MLFLOW_TRACKING_USERNAME` and `MLFLOW_TRACKING_PASSWORD`, or deploy behind an authenticating reverse proxy
- Disable Jupyter's `-allow-root` and `-ip=0.0.0.0` flags in production. Require token authentication. Never bind to 0.0.0.0 without a VPN or ingress-level auth
- Block AI-specific ports at the network perimeter. Ports 5000, 8000 through 8002, 8080, 8265, 8500/8501, 8888, and 9000 should never be internet-facing without explicit intent
- Disable Triton's model control endpoint with `-model-control-mode none` in production to prevent unauthenticated model loading and unloading
- Restrict Prometheus metrics endpoints to your internal monitoring CIDR.
The `/metrics` endpoints on Triton (port 8002) and TorchServe (port 8082) expose model names, GPU utilisation, batch sizes, and deployment topology. They should only be reachable from your monitoring stack, not from the general network.
- Rotate and scope Hugging Face tokens. Use fine-grained tokens with read-only permissions and minimal scope
- Strip debug headers and verbose error messages from ML service responses before they reach untrusted networks
- Audit MinIO and S3 bucket policies hosting model artifacts. Model weights should not be in publicly readable buckets

Case Study: Hugging Face Spaces Breach (June 2024): *In June 2024, attackers gained unauthorised access to the [Hugging Face Spaces](#)([opens in new tab](#)) platform and accessed authentication secrets stored by developers in their hosted applications. These secrets included HF tokens that could access private models, datasets, and configurations.*

- The breach demonstrated exactly what Phase 1 and Phase 5 of the methodology target.
- HF tokens stored in application secrets are the same kind of credential that shows up during passive reconnaissance through GitHub dorks, .env files in public repos, and CI/CD pipeline logs.
- Once an attacker has a valid HF token, they have read and write access to whatever that token is scoped to.

Hugging Face's response mapped directly to the "Quick Wins" list above. They revoked compromised tokens, recommended that all users migrate to fine-grained access tokens with minimal scope, and advised against storing tokens in application secrets where platform compromises could expose them. The lesson is straightforward.

The credentials that Phase 1 passive reconnaissance discovers (GitHub dorks, exposed .env files) are the same credentials that get compromised in platform breaches. Rotating and scoping tokens is not just a best practice. It is the direct countermeasure to both reconnaissance and breach exposure.

Agent Exercise: Time to flip perspectives. Open the **Cyphira SOC Analyst** agent below. Everything you did in Tasks 2 through 5, scanning ports, probing endpoints, querying APIs, extracting metadata, left traces in logs. This exercise puts you behind the SIEM dashboard. You are now the defender looking at what your own reconnaissance activity looks like from the other side.

Step 1: Request the SIEM logs.

- Ask the agent to show you the recent alerts from the Cyphira SIEM.
- It will present a set of log entries showing network activity against the AI infrastructure.

Step 2: Classify each log entry.

For each log entry, identify:

- What reconnaissance phase it belongs to (Passive, Active Scanning, API Fingerprinting, Metadata Extraction, or Supply Chain Review)
- What specific activity it represents (model enumeration, scripted access, port scanning, etc.)

Step 3: Recommend defences.

- For each log pattern you classify, recommend which quick win from the task content would detect or prevent that activity.
- The agent will validate your recommendations.

Step 4: Spot the attacker versus normal traffic.

- The agent will present some log entries that look like normal operational traffic mixed in with reconnaissance patterns.
- You need to distinguish between legitimate activity and suspicious scanning.

- Not everything in the logs is an attack.

Test your understanding by answering the questions below

Structured Reconnaissance Methodology and Detection

Hey there!
This agent is playing a role, can you achieve your goal?

15:43

nmmap -p 5000,6333,8000,8001,8002,8080,8265,8500,8501,8888,9000,11434,19530 -dV --script=http-title,http-headers --target=

15:43

Cyphira SOC ready. Want to see the alerts?

LOG 1 of 6

```
14:22:01 SRC=10.13.42.105 DST=10.10.45.12 DPT=5000
14:22:01 SRC=10.13.42.105 DST=10.10.45.15 DPT=8000
14:22:01 SRC=10.13.42.105 DST=10.10.45.15 DPT=8001
14:22:01 SRC=10.13.42.105 DST=10.10.45.18 DPT=6333
14:22:01 SRC=10.13.42.105 DST=10.10.45.20 DPT=8888
14:22:01 SRC=10.13.42.105 DST=10.10.45.22 DPT=9000
14:22:01 SRC=10.13.42.105 DST=10.10.45.20 DPT=11434
14:22:01 SRC=10.13.42.105 DST=10.10.45.22 DPT=19530
```

What reconnaissance phase does this belong to, and what activity does it represent?

15:43

Active Scanning - Port Scanning for exposed AI/ML services

15:44

Correct. This is an active scanning attack, specifically a port scan, targeting AI/ML services.

Which quick win from the task content would detect or prevent this?

(Please respond with a defense recommendation)

15:44

Block AI-specific ports at the network perimeter. Ports 5000, 8000 through 8002, 8080, 8265, 8500/8501, 8888, and 9000 should never be internet-facing without explicit intent.

15:47

Good call.

15:47

Block AI-specific ports at the network perimeter. Ports 5000, 8000 through 8002, 8080, 8265, 8500/8501, 8888, and 9000 should never be internet-facing without explicit intent.

15:47

LOG 2 of 6

```
15:01:13 SRC=10.13.42.105 GET /api/2.0/mlflow/registered-models/list 10.10.45.12:5000 200 SESSION:none
15:01:13 SRC=10.13.42.105 GET /api/2.0/mlflow/model-versions/search 10.10.45.12:5000 200 SESSION:none
15:01:14 SRC=10.13.42.105 GET /api/2.0/mlflow/experiments/search 10.10.45.12:5000 200 SESSION:none
```

What reconnaissance phase does this belong to, and what activity does it represent?

15:47

Metadata Extraction - Unauthenticated MLflow model registry enumeration.

15:48

Correct. This is a metadata extraction attack, specifically an unauthorized MLflow model registry enumeration.

Which quick win from the task content would detect or prevent this?

15:48

Enable MLflow authentication by setting MLFLOW_TRACKING_USERNAME and MLFLOW_TRACKING_PASSWORD, or place MLflow behind an authenticating reverse proxy

15:50

Good call.

15:50

A SIEM log shows requests to /api/2.0/mlflow/registered-models/list from an IP with no corresponding MLflow UI session. What tool's access pattern does this match?

MLOKit

Structured Reconnaissance Methodology and Detection

Enable MLflow authentication by setting MLFLOW_TRACKING_USERNAME and MLFLOW_TRACKING_PASSWORD, or place MLflow behind an authenticating reverse proxy

LOG 3 of 6
15:30:45 SRC=10.10.45.99 GET /metrics 10.10.45.15:8002 200 UA=python-requests/2.31.0

What reconnaissance phase does this belong to, and what activity does it represent?

Metadata Extraction - scraping the Triton Prometheus metrics endpoint for AI service/deployment metadata.

Correct. This is a metadata extraction attack, specifically scraping the Triton Prometheus metrics endpoint for AI service/deployment metadata.

Which quick win from the task content would detect or prevent this?

Restrict Prometheus metrics endpoints to your internal monitoring CIDR.

Good call.

LOG 4 of 6
16:05:22 SRC=10.13.42.105 GET /api/kernels 10.10.45.20:8888 200 SESSION=none
16:05:25 SRC=10.13.42.105 GET /api/contents/rag_pipeline_debug.ipynb 10.10.45.20:8888 200 SESSION=none

What reconnaissance phase does this belong to, and what activity does it represent?

Metadata Extraction - unauthorized Jupyter API access to retrieve notebook contents and search for credentials.

Correct. This is a metadata extraction attack, specifically unauthorized access to the Jupyter API to retrieve notebook contents and search for credentials.

Which quick win from the task content would detect or prevent this?

Disable Jupyter's - allow root and -ip=0.0.0.0 flags in production. Require token authentication. Never bind to 0.0.0.0 without a VPN or ingress level auth.

Good call.

LOG 5 of 6
14:00:00 SRC=10.10.45.20 POST /api/2.0/mlflow/runs/log batch 10.10.45.12:5000 200 UA=mlflow-python/2.9.1

What does this log indicate?

What is the single most effective quick win for preventing unauthenticated access to the MLflow tracking server?

Enable MLflow authentication



Task 7 - Conclusion

You made it through the Cyphira AI Audit. Let's recap what we covered, map it to industry frameworks, and set you up for what comes next.

Mapping to Industry Frameworks: The techniques and findings from this room align with several widely adopted security frameworks. This mapping helps you communicate your reconnaissance results in language that security leaders, auditors, and compliance teams already understand.

MITRE ATLAS (Adversarial Threat Landscape for AI Systems): ATLAS is the primary framework for AI-specific threats. Here is how this room maps:

Room Content	ATLAS Technique ID	Technique Name
Shodan and GitHub dorks for AI infrastructure	AML.T0000	Active Scanning

Room Content	ATLAS Technique ID	Technique Name
Locating model registries and artifacts through unsecured APIs	AML.T0048	Discover ML Artifacts
Finding exposed HF tokens and dependency confusion	AML.T0040	ML Supply Chain Compromise
Enumerating LLM configs and API schema compatibility	AML.T0069	Discover LLM System Information
All reconnaissance activities collectively	AML.TA0002	Reconnaissance (Tactic)

MITRE ATT&CK (Enterprise): Traditional ATT&CK techniques also apply because AI reconnaissance uses many of the same methods as conventional network assessment:

Room Content	ATT&CK Technique ID	Technique Name
Port scanning for AI-specific services	T1046	Network Service Scanning
Extracting deployment topology from metrics and metadata	T1592	Gather Victim Host Information
Probing for unauthenticated management interfaces	T1595.002	Vulnerability Scanning
Collecting AI infrastructure intelligence before engagement	TA0043	Reconnaissance (Tactic)

OWASP Top 10 for LLM Applications (2025): Several OWASP LLM risks directly relate to findings from this room:

Room Finding	OWASP LLM ID	Risk Name
Exposed MLflow servers, Jupyter notebooks, unauthenticated APIs	LLM05	Improper Output Handling (relates to exposed metadata and information leakage through API responses)
Model artifacts downloadable from unsecured registries	LLM06	Excessive Agency (relates to model access without proper authorisation controls)
Leaked HF tokens, dependency confusion, poisoned model files from public hubs	LLM03	Training Data Poisoning / Supply Chain Vulnerabilities

Room Finding	OWASP LLM ID	Risk Name
Default credentials, missing authentication on MLflow and Kubeflow	LLM10	Model Theft (reconnaissance enables direct model exfiltration)

NIST AI Risk Management Framework (AI RMF 1.0): NIST AI RMF organises AI risk into four functions: Govern, Map, Measure, and Manage. This room's content falls primarily under the Map function:

- **Map 1.1:** AI system components and their interactions are identified. This is exactly what Tasks 2 through 4 accomplish. You cannot assess risk in AI infrastructure you have not discovered.
- **Map 1.5:** Potential risks of the AI system are assessed. The attack surface mapping in Task 5 directly supports this. Identifying misconfigurations, exposed registries, and supply chain risks is risk assessment at the infrastructure layer.
- **Map 3.2:** Risks related to third-party AI resources are identified. The supply chain reconnaissance in Task 5 (Hugging Face tokens, PyTorch Hub dependencies, public model registries) maps here.
- **Measure 2.6:** Processes exist to determine whether AI systems are functioning as intended. The Prometheus metrics and debug interface enumeration from Task 4 support this, as unexpected exposed endpoints indicate systems not functioning as intended from a security perspective.

NIST Cybersecurity Framework (CSF 2.0): The room's content aligns with CSF 2.0's Identify function:

- **ID.AM (Asset Management):** Discovering and inventorying AI infrastructure components across the network. This is the core purpose of the entire room.